Chap. 4 : Systèmes de fichier Linux

1. Gestion des systèmes de fichiers :

1.1. Les systèmes de fichiers courants

Un système d'exploitation est dans la plupart des cas installé sur un disque dur ou périphérique de stockage assimilé. Pour exploiter convenablement tout ou partie de cet espace de stockage, il convient de le segmenter dans un premier temps, c'est le partitionnement, puis de créer sur les partitions à exploiter un système de fichiers.

Le système de fichier sert à organiser un espace de stockage brut, comme une partition de disque pour y stocker des données. Si le terme courant est souvent le formatage de l'espace de stockage, on parle souvent en environnement Linux de création de filesystem.

On parle de « ***filesystem*** » lorsqu'il s'agit d'un système de fichier attaché à un périphérique de stockage unique, et on parlera de « système de fichiers » pour désigner l'espace de stockage organisé, qu'il soit composé d'un ou de plusieurs *filesystem*.

Il existe plusieurs types de filesystem, dont les plus courants en environnement Linux sont : ext, reiserfs et xfs.

Ext
ext est le filesystem historique des systèmes Linux. Il existe actuellement trois versions de filesystem ext en production. ext2 est la version historique, ext3 est une évolution de ext2 qui lui ajoute un journal de transactions, et ext4 est une dernière évolution qui équipe les systèmes les plus récents et qui vise à pallier les limites de l'ext3 (taille maximum de fichiers portée de 2 téraoctets pour ext3 à 16 téraoctets pour ext4 par exemple). Le journal de transactions présent en ext3 et ext4 permet d'accélérer notablement les vérifications sur les systèmes de fichiers et la récupération en cas de crash.

Reiserfs
Reiserfs est un filesystem journalisé qui offre pour certaines opérations des performances un peu meilleures que ext3. Reiserfs se posait d'ailleurs en concurrent de ext à sa création. Ancien système de fichier par défaut des distributions Suse, reiserfs est aujourd'hui en voie de raréfaction. On lui reproche selon les conditions d'emploi une certaine fragilité ou un manque de performances globales.

Xfs
xfs est le filesystem historique des serveurs Unix. Il a été placé sous licence GPL en 2000. De bonnes performances ainsi que le support des très gros espaces de stockages (8 exaoctets de taille maximum pour le filesystem contre 16 et 32 teraoctets pour reiserfs et ext3) en font un filesystem intéressant.

1.2. Les systèmes de fichiers virtuels ou pseudo-filesystems

Un filesystem courant a pour objet de permettre l'exploitation d'un espace de stockage physique par un utilisateur ou des applications. Il existe toutefois sur les systèmes Linux des filesytem virtuels qui n'ont de réalité qu'en mémoire sans occupation d'espace sur le disque. Ils sont simplement visibles à l'utilisateur sans exploiter un quelconque espace disque.

proc
Le filesystem virtuel ***proc***, généralement monté sous le répertoire /proc permet de visualiser des éléments systèmes liés à la gestion des processus par le noyau. proc montre aussi un certain nombre d'informations systèmes liées au matériel.

*_Visualisation des informations du processeur :_

On observe ici les informations techniques liées au microprocesseur employé. Notez par exemple la vitesse réelle de l'horloge au moment de l'exécution de la commande, qui atteste de la bonne gestion de l'énergie sur un système peu sollicité.

```
Sri2@serveur:~$ cat /proc/cpuinfo
processor : 0
vendor_id : AuthenticAMD
cpu family          : 15
model               : 75
model name          : AMD Athlon(tm) 64 X2 Dual Core Processor 4200+
stepping : 2
cpu MHz             : 1000.000
cache size          : 512 KB
physical id         : 0
siblings : 2
core id             : 0
cpu cores : 2
```

sys
Le filesystem virtuel ***sys***, généralement monté sous le répertoire /sys permet de visualiser des éléments systèmes liés aux périphériques.

*_Visualisation de capacités hotplug d'un disque dur :_

De nombreux pseudo-fichiers de **/proc** et **/sys** ont un contenu limité à un seul caractère. Le chiffre 0 indique que le disque **sda** n'est pas enfichable à chaud.

```
Sri2@serveur:~$ cat /sys/block/sda/removable
O
```

1.3. Création des filesystems

Les filesystems sont créés par l'administrateur sur des espaces de stockages bruts, historiquement des partitions de disque. Ils sont ensuite vérifiés, ponctuellement par l'administrateur ou à intervalles réguliers automatiquement par le système. La création de filesystem se fait avec la commande **mkfs** dont la syntaxe est :

*_Syntaxe de la commande mkfs_ : **`mkfs -t type device`**

-t type	Précision du type de filesystem à créer. Valeurs à connaître : ext2, ext3, reiserfs, xfs.
device	Fichier spécial en mode bloc qui désigne le périphérique sur lequel créer le filesystem.

1.4. Vérification des filesystems

La vérification d'un filesystem consiste essentiellement en la vérification de cohérence entre la table des inodes du filesystem et les blocs de données correspondants. C'est-à-dire que pour chaque inode, on vérifiera que les blocs de données référencés par cet inode sont bien présents, en nombre et quantité annoncés. Pour les filesystem journalisés, une option -f oblige une vérification complète d'un filesystem semblant propre, comme par exemple un filesystem qui n'aurait pas subi d'opération d'écriture depuis sa dernière vérification réussie.

La vérification de filesystem se fait avec la commande fsck.

*Syntaxe de la commande fsck : **`fsck -t type device`**

-t type	Type du filesystem à vérifier.
device	Fichier spécial en mode bloc qui désigne le périphérique sur lequel se trouve le filesystem à vérifier.

1.5. Commandes relatives aux filesystems ext

Les syntaxes indiquées ci-dessus pour les commandes mkfs et fsck sont universelles et doivent fonctionner. Toutefois, il faut savoir que ces commandes appellent en réalité des sous-programmes (mkfs.ext2 par exemple pour mkfs -t ext2), et qu'il existe par ailleurs des commandes spécialisées qui produiront le même résultat (mke2fs est un autre équivalent de mkfs -t ext2).

Contrairement à la commande fsck, e2fsck fonctionne par défaut en mode interactif. Pour un fonctionnement en mode non interactif, elle doit être utilisée avec l'option -p. Elle vérifie alors automatiquement le filesystem sans nécessiter d'intervention de l'utilisateur.

1.6. Création de filesystem ext2 ou ext3

La commande mke2fs permet de créer directement des filesystems ext. Le format ext2 est utilisé par défaut, mais l'option -j (journal) permet de créer des structures de filesystem ext3.

- Création d'un filesystem ext2 : **`mke2fs device`**
- Création d'un filesystem ext3 : **`mke2fs -j device`**

Où ***device*** représente le fichier spécial en mode bloc qui désigne le périphérique sur lequel se trouve le filesystem à créer.

1.7. Affichage et modification des filesystems ext

La commande tune2fs permet d'afficher les paramètres d'un filesystem ext et éventuellement d'en modifier certains.

*Affichage des paramètres d'un filesystem avec tune2fs : **tune2fs -l device**

Où ***device*** représente le fichier spécial en mode bloc qui désigne le périphérique sur lequel se trouve le filesystem à vérifier.

La différence entre le format ext2 et ext3 est la présence ou non d'un journal des transactions. La commande tune2fs permet d'ajouter un journal à un filesystem ext2, et donc de le convertir en ext3.

Affichage des paramètres d'un filesystem en ext3 :

Notez que la mention ***has_journal*** figure dans la section **Filesystem features**, ce qui indique un filesystem de type ext3.

```
alpha:/dev# tune2fs -l /dev/hda1
tune2fs 1.41.3 (12-Oct-2008)
Filesystem volume name:   <none>
Last mounted on:          <not available>
Filesystem UUID:          ff700a3b-b430-49a7-ae73-bd23f363a3fc
Filesystem magic number:  0xEF53
Filesystem revision #:    1 (dynamic)
Filesystem features:      has_journal ext_attr resize_inode
dir_index filetype needs_recovery sparse_super large_file
Filesystem flags:         signed_directory_hash
Default mount options:    (none)
Filesystem state:         clean
Errors behavior:          Continue
Filesystem OS type:       Linux
Inode count:              501952
Block count:              2004100
Reserved block count:     100205
Free blocks:              1656481
Free inodes:              477768
```

*Conversion d'un filesystem ext2 en ext3 avec tune2fs : **tune2fs -j device**

Où ***device*** représente le fichier spécial en mode bloc qui désigne le périphérique sur lequel se trouve le filesystem à modifier.

1.8. Dénomination des systèmes de fichiers

Certains paramètres des systèmes de fichiers sont modifiables après leur création. Parmi ces paramètres, certains prennent de plus en plus d'importance dans les systèmes Linux modernes, et simplifieront (peut-être) les opérations de montage. Ces paramètres sont le label et l'uuid. Ils permettent de monter les filesystem locaux sans avoir à les désigner par leur fichier de bloc spécial comme ***/dev/sdb1***.

a- Le label des filesystem :

Comme son nom l'indique, le label est une étiquette qu'on attribue au filesystem pour le désigner de façon confortable. Le label doit être précisé par l'administrateur soit à la création du filesystem, soit après coup avec une commande de tuning. Les systèmes Red Hat/Fedora/Centos sont les principaux utilisateurs du label.

tune2fs -L label device	Ajoute un label au périphérique de stockage device.
Reiserfstune -l label device	Ajoute un label au périphérique de stockage device.
xfs_admin -L label device	Ajoute un label au périphérique de stockage device.

b- L'UUID des filesystem :

L'UUID, (Universally Unique Identifier) comme le label permet de désigner un périphérique de stockage par un identifiant plutôt que par son fichier de bloc spécial (/dev/sdb1 par exemple). La différence avec le label est que l'affectation de l'uuid est automatique à la création du filesystem. Il peut néanmoins être réaffecté après-coup par les commandes de tuning des filesystems. De plus en plus de systèmes généralisent l'exploitation de l'uuid. C'est le cas notamment des distributions Debian/Ubuntu.

Si on ne sait pas comment déterminer l'UUID d'un nouveau système, il est généralement créé de façon aléatoire, et sa taille (128 bits) assure son unicité.

tune2fs -U uuid device	Affectation de l'UUID uuid au périphérique de stockage device
tune2fs -U random device	Affectation d'un UUID aléatoire au périphérique de stockage device
tune2fs -U time device	Affectation d'un UUID basé sur l'heure de création au périphérique de stockage device
reiserfstune -u uuid device	Affectation de l'UUID uuid au périphérique de stockage device
xfs_admin -U uuid device	Affectation de l'UUID uuid au périphérique de stockage device

Dans le tableau ci-dessus, ***device*** représente le fichier spécial en mode bloc qui représente le périphérique hébergeant le filesystem sur lequel on intervient (Par exemple **/dev/sda3**).

2. Gestion des disques durs

Dans la plupart des situations, les connaissances courantes sur la dénomination courante des disques durs (hda, sda) sont suffisantes, et on se concentre surtout sur la façon de les exploiter sous forme de partitions ou volumes logiques. Il arrive toutefois qu'il soit nécessaire de paramétrer les disques durs du point de vue matériel, pour optimiser les performances ou pour détecter des défaillances.

2.1. Détermination des fichiers spéciaux

Les systèmes Linux contenaient dans leur répertoire /dev l'ensemble des fichiers spéciaux pour tous les périphériques gérables par le noyau. Avec le noyau 2.6 est arrivé le service ***udev***, qui a pour tâche de gérer dynamiquement la création de fichiers spéciaux à la découverte d'un périphérique.

Du point de vue de l'utilisateur ordinaire, le service ***udev*** travaille en arrière-plan : les fichiers spéciaux sont présents quand on en a besoin et il n'y a rien à vouloir de plus. En revanche, l'administrateur ou l'utilisateur avancé peut créer des règles comportementales qui permettent de déclencher des actions en fonction d'évènements liés au stockage. L'emplacement de ces règles est précisé dans le fichier de configuration de udev : ***/etc/udev/udev.conf***. En l'absence d'information, c'est la valeur par défaut qui est employée, à savoir ***/etc/udev/rules.d***.

**Exemple de règle udev :*

Dans cette configuration standard d'une distribution Debian, on constate que le système génère des liens symboliques pour les différentes appellations courantes du lecteur de media optique.

```
root@serveur # cat /etc/udev/rules.d/70-persistent-cd-rules
ENV{ID_CDROM}=="?*", ENV{ID_PATH}=="pci-0000:00:14.1-scsi-
0:0:0:0", SYMLINK+="cdrom", ENV{GENERATED}="1"
ENV{ID_CDROM}=="?*", ENV{ID_PATH}=="pci-0000:00:14.1-scsi-
0:0:0:0", SYMLINK+="cdrw", ENV{GENERATED}="1"
ENV{ID_CDROM}=="?*", ENV{ID_PATH}=="pci-0000:00:14.1-scsi-
0:0:0:0", SYMLINK+="dvd", ENV{GENERATED}="1"
ENV{ID_CDROM}=="?*", ENV{ID_PATH}=="pci-0000:00:14.1-scsi-
0:0:0:0", SYMLINK+="dvdrw", ENV{GENERATED}="1"
```

2.2. Informations sur les périphériques de stockage

Grâce au service ***udev***, on ne trouve plus dans le répertoire /dev que les périphériques réellement présents sur le système. Cela constitue naturellement un premier niveau d'informations.

La solution la plus simple pour obtenir plus de détails est d'exploiter la commande ***dmesg*** qui consigne tous les messages renvoyés par le noyau depuis son démarrage. On dit que la commande dmesg affiche le **ring-buffer** du noyau.

a)Utilisation de dmesg pour identifier les disques durs :

Il est vivement recommandé de filtrer la sortie de la commande dmesg par le filtre grep :

```
alpha:~# dmesg | grep [sh]d
[    0.000000] Kernel command line: root=/dev/hda1 ro quiet
[    3.136965] hda: VBOX HARDDISK, ATA DISK drive
[    3.822425] hda: host max PIO4 wanted PIO255(auto-tune)
selected PIO4
[    3.822677] hda: UDMA/33 mode selected
[    4.575784] hdc: VBOX CD-ROM, ATAPI CD/DVD-ROM drive
[    5.275977] hdc: host max PIO4 wanted PIO255(auto-tune)
selected PIO4
[    5.275977] hdc: UDMA/33 mode selected
[    7.203721] hda: max request size: 128KiB
[    7.203728] hda: 16777216 sectors (8589 MB) w/256KiB Cache,
```

```
CHS=16644/16/63
[    7.204020] hda: cache flushes supported
[    7.204020]  hda: hda1 hda2 < hda5 >
[    7.234912] hdc: ATAPI 32X DVD-ROM drive, 128kB Cache
[    7.257272] Driver 'sd' needs updating - please use bus_type methods
[    7.257525] sd 0:0:0:0: [sda] 4194304 512-byte hardware sectors
(2147 MB)
[    7.257620] sd 0:0:0:0: [sda] Write Protect is off
[    7.257627] sd 0:0:0:0: [sda] Mode Sense: 00 3a 00 00
[    7.257769] sd 0:0:0:0: [sda] Write cache: enabled, read cache:
enabled, doesn't support DPO or FUA
(...)
```

b)Récupération d'informations sur un périphérique par la commande udevadm

Le service ***udev*** peut aussi nous fournir des informations précieuses par le biais de sa commande d'administration ***udevadm***.

```
serveur:~# udevadm info --query=all --name=/dev/hda
P: /block/hda
N: hda
S: block/3:0
S: disk/by-id/ata-VBOX_HARDDISK_VBf92d3e4d-7faf607b
S: disk/by-path/pci-0000:00:01.1-ide-0:0
E: ID_TYPE=disk
E: ID_MODEL=VBOX_HARDDISK
E: ID_SERIAL=VBf92d3e4d-7faf607b
E: ID_REVISION=1.0
E: ID_BUS=ata
E: ID_PATH=pci-0000:00:01.1-ide-0:0
```

c)Récupération d'informations du répertoire /dev/disk

On trouve dans le répertoire **/dev/disk** les éléments de stockage apparaissant selon la façon dont ils sont reconnus et identifiés par le système.

```
root@serveur:/dev/disk$ ls
by-id  by-label  by-path  by-uuid
root@serveur:/dev/disk$ cd by-uuid
root@serveur:/dev/disk/buy-uuid$ ls
52200c0b-aee8-4ae0-9492-1f488051e4a3  B0F82CDCF82CA318
b0891c0e-1812-4d23-b77d-b861f7fd2713  ee7890fb-c312-406f-b100-669c97ee8d07
root@serveur:/dev/disk/by-uuid$ file *
52200c0b-aee8-4ae0-9492-1f488051e4a3: symbolic link to `../../sda2'
b0891c0e-1812-4d23-b77d-b861f7fd2713: symbolic link to `../../sdb1'
B0F82CDCF82CA318:                     symbolic link to `../../sda1'
ee7890fb-c312-406f-b100-669c97ee8d07: symbolic link to `../../sda3'
root@serveur:/dev/disk/by-uuid$
```

2.3. Gestion des partitions avec fdisk

2.4. Gestion des défaillances matérielles

Si une incohérence est due à un problème de gestion de l'écriture (arrêt électrique lors d'une opération d'écriture), ***fsck*** peut essayer de réparer l'erreur. En revanche, la défaillance physique d'un disque, liée à un défaut de la surface magnétique par exemple, doit être traitée

de façon adéquate avec la commande ***badblocks***. La commande ***badblocks*** référence les blocs physiquement défectueux sur un disque ou une partition et les envoyent sur la sortie standard, mais il est courant d'utiliser un fichier qui sera exploitable par les programmes ***e2fsck*** ou ***mke2fs***. Dans ce cas, il faut préciser la taille des blocs employés mais si le but est simplement de vérifier l'absence de défaut sur le disque, la commande ***badblocks*** peut être utilisée sans aucune option.

**Détection des blocs en erreur avec badblocks :*

Syntaxe : **`badblocks -b taille_blocks -o fichier_sortie  device`**

Où ***taille_blocks*** représente la taille des blocs du système de fichiers et ***fichier_sortie*** le fichier qui consignera l'ensemble des blocs altérés (défectueux).

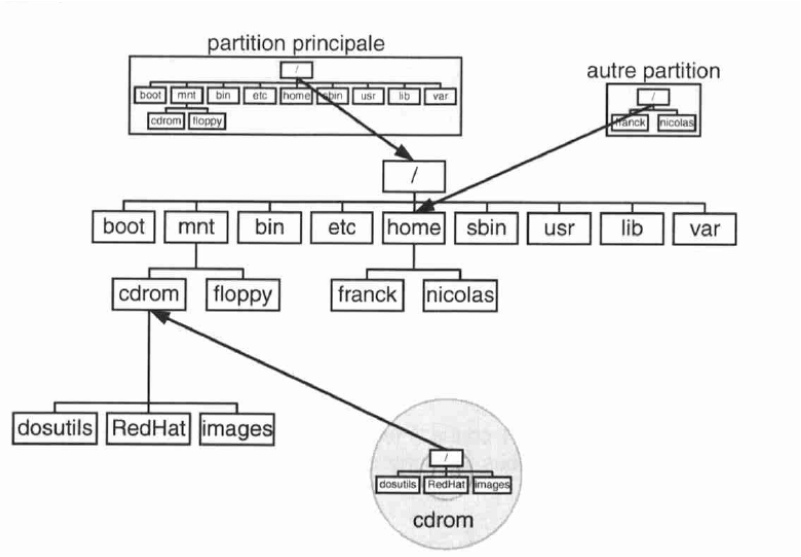
Et ***device*** représente le fichier spécial en mode bloc qui désigne le périphérique sur lequel se trouve le filesystem.

3.Montage des filesystems

3.1. Montage et démontage

La commande ***mount*** permet de monter le filesytem d'un périphérique de stockage sous un répertoire local, généralement vide. Au minimum, il faut fournir comme argument à la commande mount le périphérique hébergeant le filesystem, et le répertoire qui constituera son point de montage.

La commande ***umount*** réalise l'opération inverse : démontage. Elle accepte comme argument le point de montage, ou le périphérique physique à démonter.



3.1.a.Syntaxe de la commande mount :

`mount -t type_fs -options device  point_montage`

type_fs	Facultatif : type de filesystem à monter.
options	Facultatif : options de montage.
device	Le périphérique hébergeant un filesystem à monter, sous forme de fichier spécial bloc.
point_montage	Le répertoire qui servira de point d'ancrage au filesystem monté.

Les options de montage sont :

- ro (lecture seule)
- sync (écritures synchrones sans passer par un cache mémoire)
- loop (montage de données de fichiers plutôt que de filesystems).

Le montage d'un filesystem avec l'option ***sync*** permet de s'affranchir de toute forme de cache en écriture sur le disque, et ainsi de fiabiliser les opérations d'écriture.

3.1.b. Syntaxe de la commande umount :

```
umount -options device point_montage
```

options	Facultatif : options de démontage.
device	Facultatif si le point de montage est précisé : le périphérique à démonter.
point_montage	Facultatif si le périphérique est précisé : le répertoire servant de point de montage à libérer.

Les options de démontage sont :

- -f (force : forcer le démontage)
- -l (lazy : démontage lent qui sera effectif quand toutes les ressources utilisées pour le montage auront pu être libérées.

Le démontage d'un filesystem est indispensable pour en effectuer la vérification avec la commande e2fsck. Le filesystem monté sur / est par définition indémontable puisque toujours utilisé. Il est possible de forcer la vérification avant le montage lors du démarrage depuis la commande shutdown.

Pour vérifier le filesystem racine avant montage, on doit executer la commande suivante :

```
shutdown -F -r now
```

3.1.c. Visualisation des filesystems montés

La commande **mount** sans argument permet de visualiser les filesystems montés. Par ailleurs, chaque montage réussi provoque l'écriture d'une ligne correspondante dans le fichier ***/etc/mtab***. L'affichage du fichier ***/proc/mounts*** renvoie les mêmes informations.

3.2. Fichier fstab

Le fichier ***/etc/fstab*** permet de désigner des filesystems à monter ou des espaces de swap à activer automatiquement au démarrage. Il permet aussi de désigner des filesystems éventuellement montables, comme pour les périphériques amovibles par exemple.

Le fichier **/etc/fstab** doit comporter sur chaque ligne l'ensemble des éléments nécessaires au montage d'un filesystem, à savoir : le point de montage, la désignation de l'espace de stockage, et les options de montage.

N.B : Pour les espaces de swap, la désignation du point de montage sera sans objet.

3.2.1. Structure du fichier /etc/fstab

Le fichier ***/etc/fstab*** est composé d'une ligne par filesystem à monter, et chaque ligne étant composée de six champs obligatoires qui sont :

```
fs  pointmontage  type  options  dump  fsck
```

Les six champs sont séparés par des espaces ou des tabulations et dont la description est :

Numéro de champ	Champ	Description
1	fs	Filesystem, désigné par son fichier de bloc spécial, son label ou son uuid.
2	pointmontage	Point de montage.
3	type	Type de filesystem qui est obligatoirement **swap** pour le swap, **auto** ou **type effectif** de filesystem dans le cas contraire.
4	options	Options de montage qui sont admises par la commande mount.
5	dump	Facultatif : Si la commande dump est utilisée pour la sauvegarde du système, ce champ doit être à 1 pour assurer la sauvegarde. Sinon, sa valeur par défaut est 0.
6	fsck	Facultatif : En cas de vérification automatique des filesystem au démarrage, indique dans quel ordre cette vérification doit se faire. Les valeurs obligatoires possibles de ce champ sont : o 1 ➔ pour le filesystem monté sur / o 2 ➔ pour les autres o 0 ➔ pour que la vérification ne soit jamais effectuée.

**Exemple de fichier /etc/fstab sur Debian :*

```
# /etc/fstab: static file system information.

# <file system> <mount point>   <type>  <options>       <dump>  <pass>
```

```
proc            /proc           proc    defaults          0       0
/dev/sda1       /               ext3    errors=remount-ro 0       1
/dev/sda5       none            swap    sw                0       0
/dev/hdc        /media/cdrom0   udf,iso9660 user,noauto   0       0
```

3.2.2. Options de montage du fichier /etc/fstab

Les principales options disponibles pour le montage sont les suivantes :

Option	***Description***
`defaults`	permet de choisir les options par défaut pour ce système de fichiers
`auto`	permet au système de fichiers qu'il soit monté automatiquement au démarrage du système
`user`	permet d'autoriser le montage de ce système de fichiers par les utilisateurs
`ro`	permet de monter le système de fichiers en lecture seule
`rw`	permet de monter le système de fichiers en lecture et écriture
`exec`	permet d'autoriser l'exécution des fichiers exécutables sur ce système de fichiers si celui-ci ne supporte pas la notion de droit d'exécution
`acl`	permet d'autoriser l'utilisation des ACLs (« Access Control List ») pour fixer les droits des utilisateurs sur les fichiers
`uid=utilisateur`	permet de spécifier le numéro ***`utilisateur`*** de l'utilisateur propriétaire du répertoire racine de ce système de fichiers
`gid=groupe`	permet de spécifier le numéro ***`groupe`*** du groupe d'utilisateurs auquel le répertoire racine du système de fichiers appartient
`mode=valeur`	permet de fixer les droits sur le répertoire racine du système de fichiers à monter. La valeur ***`valeur`*** est donnée en octal comme **644**
`umask=valeur`	permet de fixer les droits sur les fichiers qui ne sont pas gérés par le système de fichiers. La valeur `valeur` est donnée en octal comme **022**

Remarque :

Quand la commande ***mount -a*** est appelée au démarrage d'un système, elle provoque le montage de tous les périphériques référencés dans le fichier /etc/fstab, à l'exception de ceux qui présentent l'option ***noauto*** dans le quatrième champ.

4. Gestion du swap

4.1. Rôle du swap

Le swap ou mémoire virtuelle est un espace de stockage exploité pour palier à un manque de mémoire physique sur le système. Quand la mémoire physique devient saturée pour les applications, une partie des informations stockées en mémoire et n'ayant pas fait l'objet d'une utilisation récente est déplacée sur l'espace de swap, libérant ainsi de l'espace pour les

applications qui ont un besoin immédiat de mémoire. Si des applications ont besoin des informations qui ont été basculées sur l'espace de swap, le mécanisme de swap est à nouveau engagé pour libérer encore de l'espace en mémoire physique, espace dans lequel les données swappées à nouveau nécessaires seront restituées pour exploitation par les applications.

En général, les installations par défaut des distributions proposent généralement la création d'un espace de swap automatiquement. Pour une installation optimale, les valeurs courantes (une à deux fois la RAM) sont recommandées.

4.2. Optimisation du swap

Le swap est optimisable en quantité c'est-à-dire qu'il peut arriver que le swap ait été sous-dimensionné à l'installation : par exemple, on installe sur un serveur existant une application qui exige une certaine quantité de RAM et un swap dix fois supérieur au swap existant.

Par ailleurs, le swap peut être déplacé vers un espace disque plus rapide (baie de disque récente) et donc plus rapide que le système disque initial est installée, et l'exploitation du swap pourrait être plus rapide sur ces systèmes de stockage. Pour ces raisons, il peut être utile de créer un nouvel espace de swap, qui s'ajoutera ou se substituera à l'espace initial du swap.

4.2.1. Nature de l'espace de swap

Le swap peut être constitué de plusieurs espaces de stockage qui sont des partitions ou des fichiers. Dans la mesure où le noyau accèdera directement et exclusivement aux partitions de swap, les performances seront meilleures qu'avec un fichier de swap où le filesystem représente un intermédiaire supplémentaire vers le stockage physique.

- Si le swap est placé sur une partition, elle doit avoir été créée de type 82 avec un outil de partitionnement adéquat (fdisk Linux par exemple).
- Si c'est un fichier, il doit simplement être accessible en permanence sur un filesystem toujours monté.

4.2.2. Création de l'espace de swap

La création du swap s'effectue avec la commande ***mkswap***, et elle peut être appliquée aussi bien à une partition qu'à un fichier de taille déterminée.

Syntaxe* : **`mkswap espace_stockage`

Où ***espace_stockage*** représente l'emplacement physique de l'espace de swap dont la dénomination peut se faire de différentes façons :

/chemin/fichier	Structure le fichier afin qu'il puisse être exploité en tant qu'espace de swap.
/dev/device	Structure l'espace de stockage désigné par le fichier spécial en mode bloc afin qu'il puisse être exploité en tant qu'espace de swap.
-L LABEL	Structure l'espace de stockage désigné par le label LABEL afin qu'il puisse être exploité en tant qu'espace de swap.

-U UUID	Structure l'espace de stockage désigné par l'uuid UUID afin qu'il puisse être exploité en tant qu'espace de swap.

4.2.3. Activation d'un espace de swap

Une fois l'espace de swap créé, il doit être rendu accessible au noyau par la commande ***swapon***. Le système sera alors capable de swapper à partir du nouvel espace créé.

*Syntaxe : **swapon espace_stockage**

Où ***espace_stockage*** représente l'emplacement physique de l'espace de swap dont la dénomination peut se faire de différentes façons (voir tableau ci-dessus).

4.2.4. Désactivation d'un espace de swap

Si on souhaite que le système arrête d'exploiter un espace de swap, il faut le lui signifier avec la commande ***swapoff***.

*Syntaxe : **swapoff espace_stockage**

Où ***espace_stockage*** représente l'emplacement physique de l'espace de swap dont la dénomination peut se faire de différentes façons (voir tableau ci-dessus).

4.2.5.Visualisation des espaces de swap

L'ensemble des espaces de swap exploités, ainsi que leur nature (fichier ou partition) peuvent être affichés avec la commande ***swapon***.

*Syntaxe : **swapon -s**

*Exemple d'utilisation de la commande swapon :

```
Debian:~# swapon -s
Filename    Type        Size      Used  Priority
/dev/hda5   partition   409616    608  -1
```

Cette commande indique la partition ou le fichier utilisé, la taille réservée et la quantité de swap utilisée.

Remarque:
Il est également possible de visualiser la configuration du swap en consultant le contenu du fichier swap du filesystem virtuel ***/proc/swaps***.